

Whack a Mole Game

1. Introduction

- The **Whack-A-Mole game** is a popular reaction-based arcade game in which players must quickly respond to randomly appearing targets. In this project, the concept of the game is implemented using **Field Programmable Gate Array (FPGA)** technology.
- The system uses **LEDs to represent the moles**, which appear randomly on the board. The player must press the **corresponding switch** to hit the active LED.
- When the correct switch is pressed while the LED is active, the **player's score increases and is displayed on the 7-segment (HEX) display**. The game runs for a **30-second duration**, which is also displayed on the 7-segment display as a countdown timer.
- At the end of the game, a **special LED blinking pattern is displayed** to indicate that the game has finished. Additionally, the **LCD display shows a welcome message along with the names**.
- This project demonstrates the implementation of various digital system concepts such as **timers, counters, random LED generation, score calculation, input detection, and display interfacing using FPGA hardware**.

2. Working Principle

- The Whack-A-Mole game operates using digital logic implemented inside the FPGA. The system starts when the user presses the **start key**, which activates the game logic and initializes both the **timer and the score counter**.
- After the game begins, a **random LED is activated** to represent a mole appearing. The player must press the **corresponding switch** that matches the active LED. If the correct switch is pressed while the LED is active, the system increments the **score**, which is then displayed on the **HEX (7-segment) display**.
- At the same time, a **30-second countdown timer** runs continuously and is also displayed on the **7-segment display**. During this period, LEDs continue to appear randomly, requiring the player to react quickly.
- The game continues until the **timer reaches zero**. Once the time expires, the game stops automatically and the final score remains displayed. After the game ends, a **LED blinking pattern animation** is shown to indicate the completion of the game.
- Additionally, the FPGA board's **LCD display shows a welcome message along with the developer names**, improving the user interaction with the system.

3. Codes

a) top.v

```
module top(

input CLOCK_50,
input [17:0] SW,

output [17:0] LEDR,
output [6:0] HEX7,
output [6:0] HEX6,
output [6:0] HEX5,
output [6:0] HEX4,
output [6:0] HEX3,
output [6:0] HEX2,
output [6:0] HEX1,
output [6:0] HEX0,

output [7:0] LCD_DATA,
output LCD_RS,
output LCD_RW,
output LCD_EN,
output LCD_ON,
output LCD_BLON

);

wire game_over;
wire start;

assign start = SW[17];
assign HEX3 = 7'b1111111;
assign HEX2 = 7'b1111111;
assign HEX1 = 7'b1111111;
assign HEX0 = 7'b1111111;

// TIMER
timer_30s t1(
    .CLOCK_50(CLOCK_50),
    .start(start),
    .HEX7(HEX7),
    .HEX6(HEX6),
    .game_over(game_over)
);
```

```
// RANDOM LED GENERATOR
random_led r1(
    .CLOCK_50(CLOCK_50),
    .start(start),
    .game_over(game_over),
    .LEDR(LEDR)
);

// SCORE MODULE
score s1(
    .CLOCK_50(CLOCK_50),
    .start(start),
    .game_over(game_over),
    .SW(SW),
    .LEDR(LEDR),
    .HEX5(HEX5),
    .HEX4(HEX4)
);

// LCD DISPLAY
lcd_display lcd(
    .CLOCK_50(CLOCK_50),
    .LCD_DATA(LCD_DATA),
    .LCD_RS(LCD_RS),
    .LCD_RW(LCD_RW),
    .LCD_EN(LCD_EN),
    .LCD_ON(LCD_ON),
    .LCD_BLON(LCD_BLON)
);

endmodule
```

b) timer_30s.v

```
module timer_30s(

input CLOCK_50,
input start,

output [6:0] HEX7,
output [6:0] HEX6,
output reg game_over

);

reg [25:0] count = 0;
reg one_sec = 0;

reg [5:0] timer = 30;

// 1 second generator
always @(posedge CLOCK_50)
begin
    if(count == 50_000_000-1)
    begin
        count <= 0;
        one_sec <= 1;
    end
    else
    begin
        count <= count + 1;
        one_sec <= 0;
    end
end
end
```

```
// timer countdown
always @(posedge CLOCK_50)
begin
    if(!start)
        timer <= 30;

    else if(one_sec && timer > 0)
        timer <= timer - 1;
end

// game over
always @(posedge CLOCK_50)
begin
    if(timer == 0)
        game_over <= 1;
    else
        game_over <= 0;
end

// split digits
wire [3:0] tens;
wire [3:0] ones;

assign tens = timer / 10;
assign ones = timer % 10;

seven_seg s1(tens, HEX7);
seven_seg s2(ones, HEX6);

endmodule
```

c) random_led.v

```

module random_led(
input CLOCK_50,
input start,
input game_over,
output reg [17:0] LEDR
);

// slow clock for random LED (1 sec)
reg [26:0] slow_counter = 0;
reg slow_tick = 0;

always @(posedge CLOCK_50)
begin
    if(slow_counter == 50_000_000)
    begin
        slow_counter <= 0;
        slow_tick <= 1;
    end
    else
    begin
        slow_counter <= slow_counter + 1;
        slow_tick <= 0;
    end
end

// fast clock for animation
reg [22:0] fast_counter = 0;
reg fast_tick = 0;

always @(posedge CLOCK_50)
begin
    if(fast_counter == 5_000_000)
    begin
        fast_counter <= 0;
        fast_tick <= 1;
    end
    else
    begin
        fast_counter <= fast_counter + 1;
        fast_tick <= 0;
    end
end

reg [4:0] lfsr = 5'b10101;

```

```

wire feedback;
assign feedback = lfsr[4] ^ lfsr[2];

always @(posedge CLOCK_50)
begin
    if(slow_tick)
        lfsr <= {lfsr[3:0], feedback};
    end

reg [3:0] step = 0;
reg [1:0] cycle = 0;
reg mode = 0;
reg final_hold = 0;

always @(posedge CLOCK_50)
begin

// RESET WHEN GAME RESTARTS
if(!start)
begin
    LEDR <= 0;
    step <= 0;
    cycle <= 0;
    mode <= 0;
    final_hold <= 0;
end

// NORMAL GAME
else if(!game_over)
begin
    if(slow_tick)
        LEDR <= 18'b1 << (lfsr % 17);
    End

// GAME OVER ANIMATION
else
begin

    if(final_hold)
        LEDR <= 18'b111111111111111111;

    else if(fast_tick)
    begin

```

```

// CENTER EXPANSION
if(mode == 0)
begin
    case(step)
        0: LEDR <= (1<<9)|(1<<8);
        1: LEDR <= (1<<10)|(1<<7);
        2: LEDR <= (1<<11)|(1<<6);
        3: LEDR <= (1<<12)|(1<<5);
        4: LEDR <= (1<<13)|(1<<4);
        5: LEDR <= (1<<14)|(1<<3);
        6: LEDR <= (1<<15)|(1<<2);
        7: LEDR <= (1<<16)|(1<<1);
        8: LEDR <= (1<<17)|(1<<0);
    endcase

    step <= step + 1;

    if(step == 8)
    begin
        step <= 0;
        cycle <= cycle + 1;
    end

    if(cycle == 2)
    begin

```

```

        cycle <= 0;
        mode <= 1;
    end
end

// ALL LED BLINK
    else if(mode == 1)
    begin
        LEDR <= ~LEDR;

        step <= step + 1;

        if(step == 4)
        begin
            final_hold <= 1;
            LEDR <=
18'b11111111111111111111;
        end
    end
end
end

endmodule

```

d) seven_seg.v

```

module seven_seg(
    input [3:0] digit,
    output reg [6:0] seg
);

always @(*)
begin
    case(digit)
        4'd0: seg = 7'b1000000;
        4'd1: seg = 7'b1111001;
        4'd2: seg = 7'b0100100;
        4'd3: seg = 7'b0110000;

```

```

        4'd4: seg = 7'b0011001;
        4'd5: seg = 7'b0010010;
        4'd6: seg = 7'b0000010;
        4'd7: seg = 7'b1111000;
        4'd8: seg = 7'b0000000;
        4'd9: seg = 7'b0010000;
        default: seg = 7'b1111111;
    endcase
end

endmodule

```

e) score.v

```
module score(  
  
    input CLOCK_50,  
    input start,  
    input game_over,  
    input [17:0] SW,  
    input [17:0] LEDR,  
  
    output [6:0] HEX5,  
    output [6:0] HEX4  
  
);  
  
    reg [7:0] score = 0;  
    reg [17:0] SW_prev = 0;  
  
    integer i;  
  
    always @(posedge CLOCK_50)  
    begin  
        if(!start)  
            score <= 0;
```

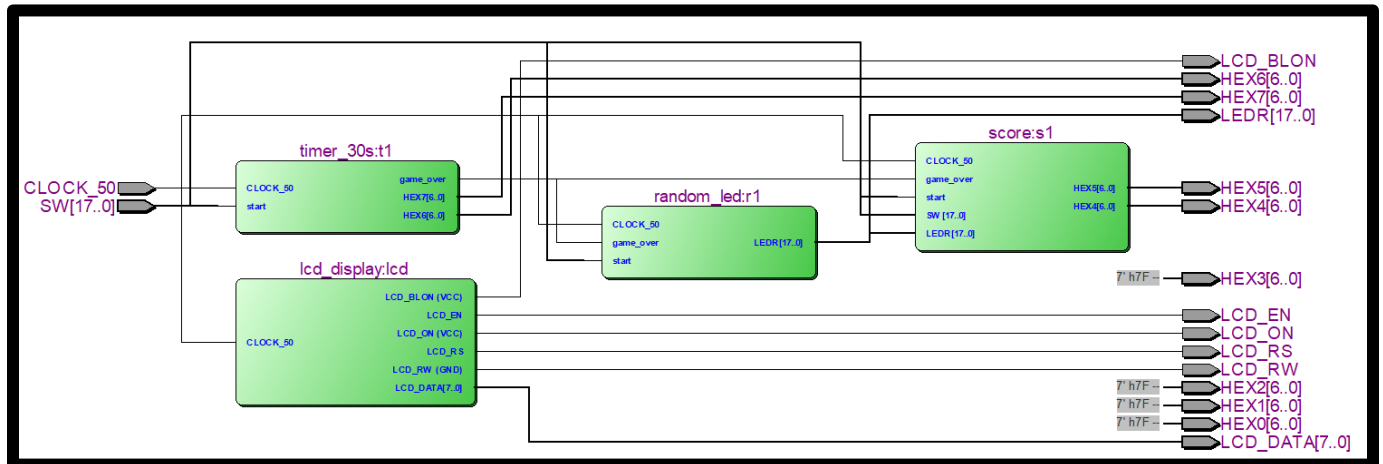
```
    else if(!game_over)  
        begin  
            for(i=0;i<18;i=i+1)  
                begin  
                    if(SW[i] && !SW_prev[i] &&  
LEDR[i])  
                        score <= score + 1;  
                end  
            end  
            SW_prev <= SW;  
        end  
  
        wire [3:0] tens;  
        wire [3:0] ones;  
  
        assign tens = score / 10;  
        assign ones = score % 10;  
  
        seven_seg s1(tens, HEX5);  
        seven_seg s2(ones, HEX4);  
  
    endmodule
```

- The Whack-A-Mole game is implemented using multiple Verilog modules. Each module performs a specific task and all modules are connected through the top-level module to form the complete system.
- **top.v** – Main module that integrates all modules and controls the overall game operation.
- **random_led.v** – Generates random LED blinking to represent the moles.
- **timer_30s.v** – Implements the 30-second countdown timer displayed on the HEX display.
- **score.v** – Calculates and updates the player's score when the correct LED is hit.
- **seven_seg.v** – Drives the 7-segment display for showing timer and score.
- **lcd_display.v** – Displays the welcome message and developer names on the LCD.

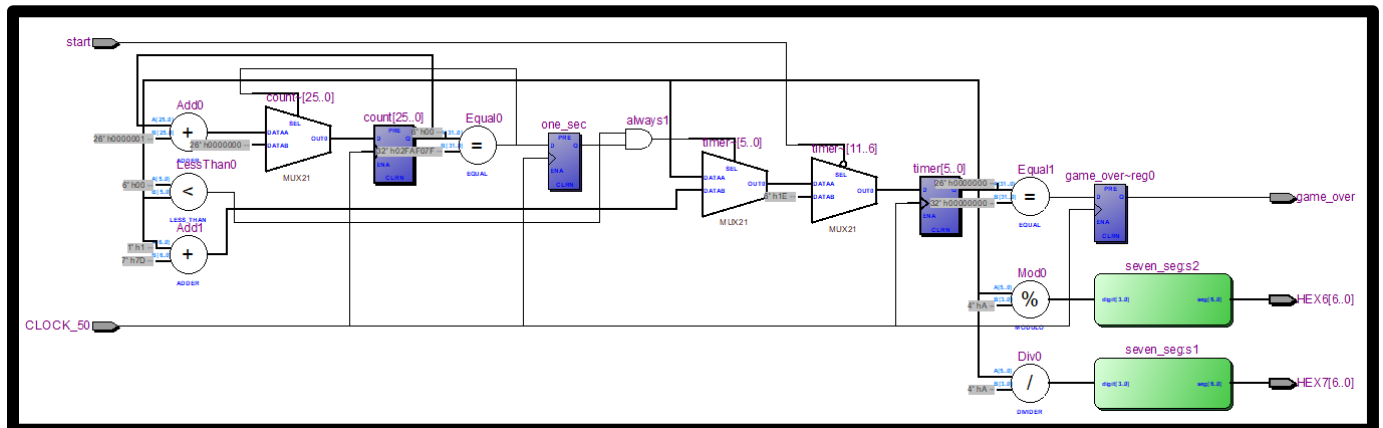
4.RTL Design

- The RTL (Register Transfer Level) design represents the internal architecture of the system and shows how different modules are interconnected in the FPGA design. The RTL schematic is generated automatically by Quartus after compiling the Verilog code.
- In this project, the RTL design consists of several modules such as **top module**, **timer module**, **random LED generator**, **score module** and **LCD display module**. The **top module** acts as the **main controller** and connects all other modules together.
- The RTL diagram helps in understanding the **data flow and connection between different modules** in the design.

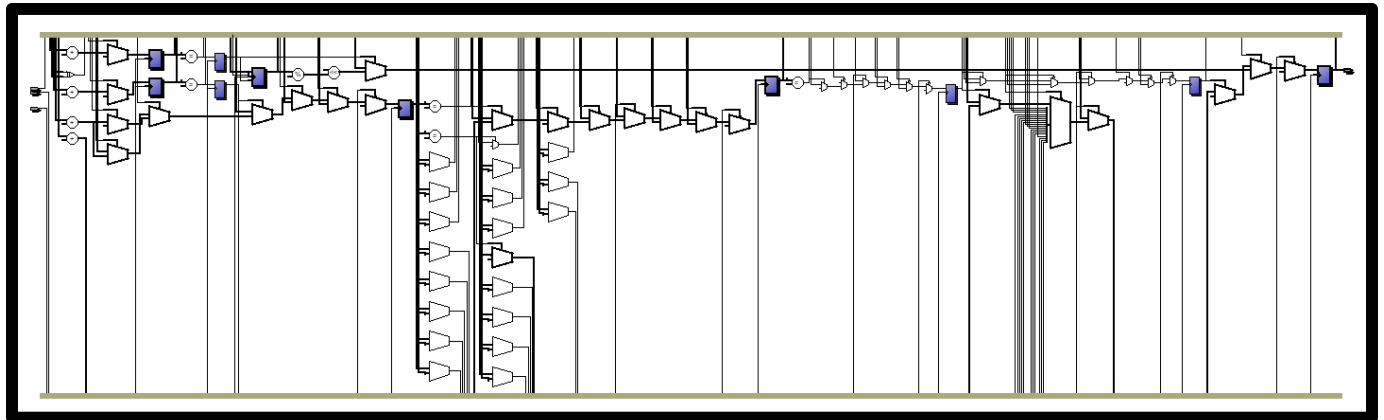
Overall RTL:



Timer RTL:



Random LED RTL:



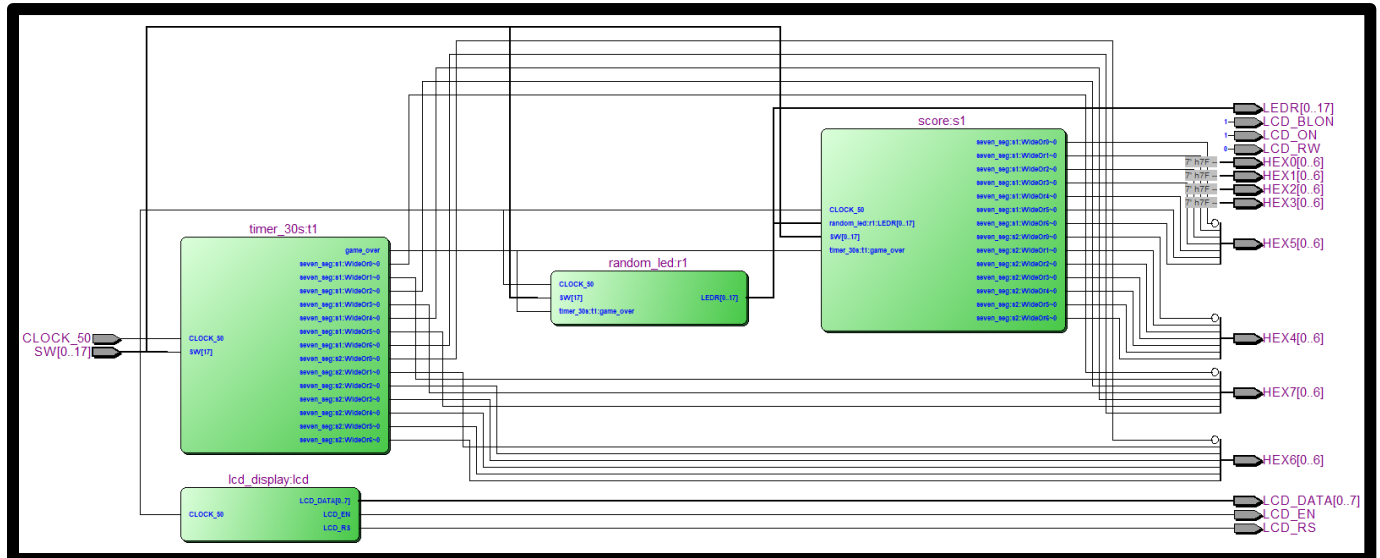
Score RTL:



5. TTL Design

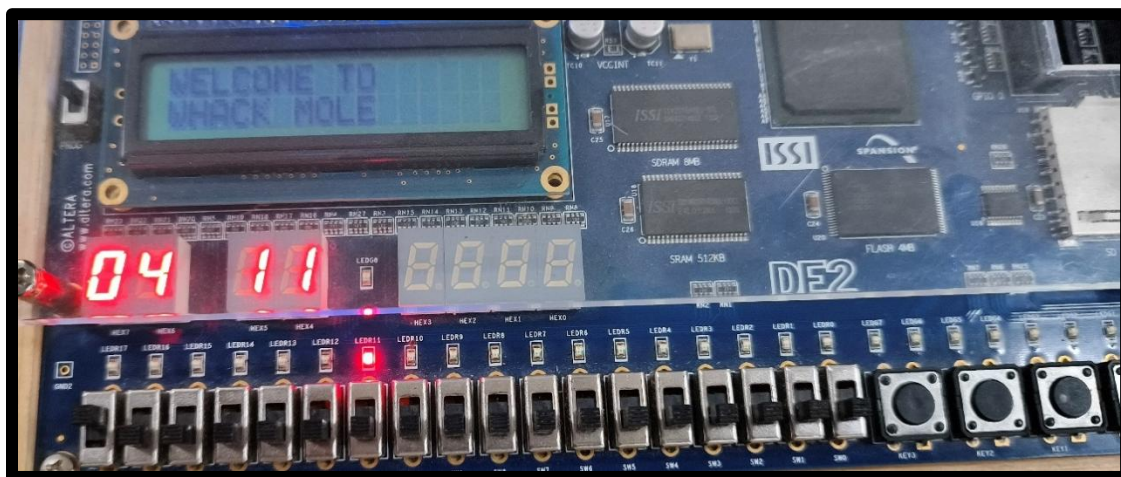
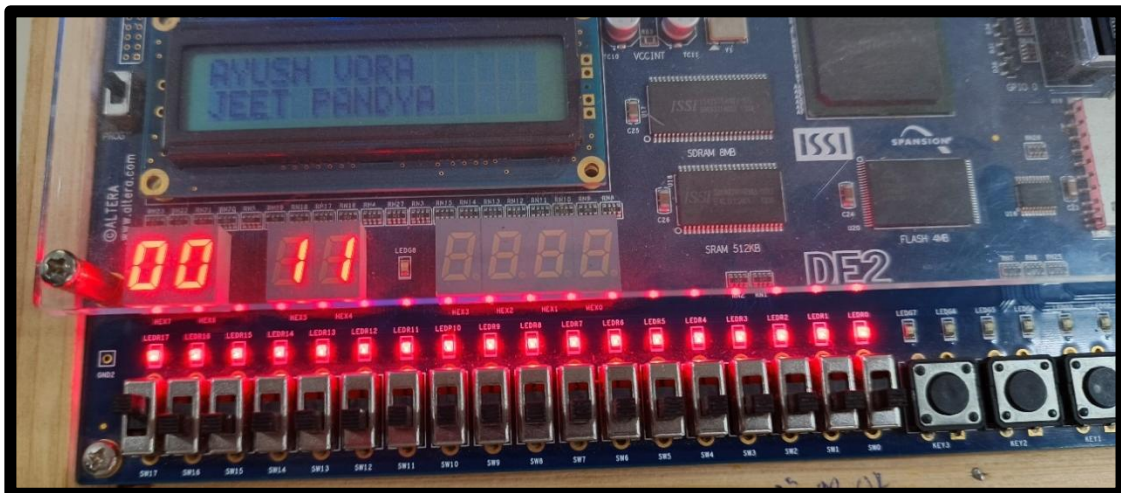
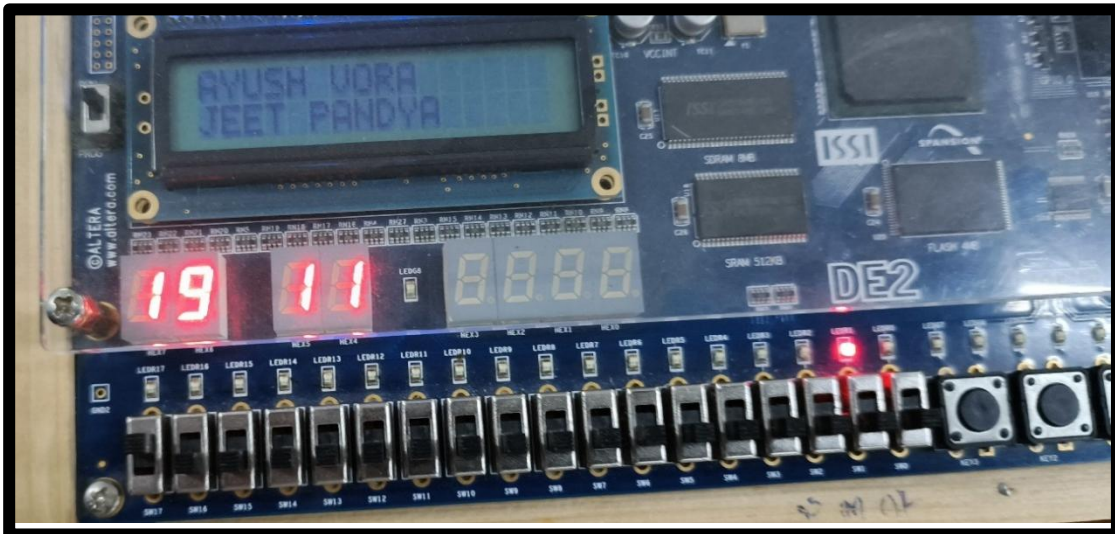
- The TTL (Transistor-Transistor Logic) design represents the **gate-level implementation** of the system. It shows how the FPGA synthesizer converts the Verilog code into basic digital components such as **logic gates, flip-flops, multiplexers, and counters**.
- In this project, the Quartus synthesis tool generates the TTL design automatically after compilation. The design internally uses **flip-flops for registers and counters**, **combinational logic for decision making**, and **clock-based logic for timing operations**.
- The TTL diagram helps in understanding the **hardware-level implementation of the Whack-A-Mole game inside the FPGA**.

Overall TTL:



6. Results

- Demo Video Link: [Whack a Mole Game](#)



7. Conclusion

- In this project, a **Whack-A-Mole game was successfully designed and implemented using FPGA technology**. The system integrates various digital components including timers, counters, random number generation, LED control, switch input detection, and display interfacing.
- The project helped in understanding the practical implementation of **digital design concepts using Verilog and FPGA hardware**. It also demonstrated how FPGA can be used to implement interactive real-time systems.
- The successful operation of the game confirms the correctness of the design and highlights the advantages of FPGA in rapid prototyping and digital system development.